

Painted Wolf Optimization: análisis de las ecuaciones de movimiento en el dominio continuo

Maverick Gayoso, Rogelio González

Magíster en Ingeniería Informática, Pontificia Universidad Católica de Valparaíso

MII902 Optimización Estocástica

Profesor: Dr. Broderick Crawford Labrín

RESUMEN: Painted Wolf Optimization (PWO) es una metaheurística poblacional estocástica propuesta en 2026 e inspirada en la exploración cooperativa, la jerarquía y el rally previo a la caza de los licaones africanos. El presente informe de avance estudia el mecanismo continuo del algoritmo y, en particular, la forma en que asigna nuevos valores reales a las variables de decisión. Cada agente representa una solución candidata y cada dimensión de su posición corresponde a una variable del problema. En cada iteración, PWO evalúa la población, actualiza la mejor solución observada y calcula una fuerza de rally. La comparación de esa fuerza con un umbral estocástico selecciona una de tres reglas de movimiento: dos estrategias de exploración y una de explotación. El trabajo formaliza estas reglas con una nomenclatura común, presenta un ruteo numérico de una iteración y contrasta la formulación publicada con las implementaciones en Python y MATLAB del repositorio de los autores. Una prueba preliminar sobre la función Sphere muestra que el código Python reduce el fitness en un escenario controlado, aunque no constituye una reproducción de los resultados del artículo. El análisis también identifica diferencias entre paper, pseudocódigo y código que requieren decisiones explícitas antes de la fase experimental. La binarización y la aplicación a un problema discreto se reservan para el informe final.

Palabras clave: optimización estocástica, metaheurística poblacional, Painted Wolf Optimization, ecuación de movimiento, exploración, explotación.

1. Introducción

Los problemas de optimización pueden presentar espacios de búsqueda extensos, funciones no convexas, múltiples óptimos locales o restricciones que dificultan la enumeración exhaustiva. Las metaheurísticas abordan estos escenarios mediante reglas de búsqueda aproximada. En los métodos poblacionales, varias soluciones candidatas exploran simultáneamente el espacio y comparten información durante el proceso.

Painted Wolf Optimization (PWO) fue presentado por Sheikhi en 2026 como una metaheurística poblacional inspirada en el comportamiento social y de caza del licaón africano [1]. El artículo propone un rally de votación para decidir si la población continúa explorando o activa una actualización con referencia en la mejor solución conocida.

Además de funciones benchmark, el autor reporta aplicaciones en problemas de diseño de ingeniería y en el entrenamiento de un sistema de detección de intrusiones. Estos resultados son antecedentes del método, pero todavía no han sido reproducidos independientemente en este proyecto.

El objetivo de este avance es responder una pregunta operacional:

¿Cómo obtiene PWO el valor de la variable j de la solución i en la iteración $t + 1$?

La pregunta obliga a separar la metáfora biológica de la mecánica de optimización. En este informe, un lobo es una solución, una posición es un vector de valores reales, una dimensión es una variable de decisión y el movimiento es una asignación iterativa. Sobre esa base se estudian la inicialización, el rally, las tres reglas de actualización y el manejo del dominio.

Las contribuciones de este avance son:

1. una notación unificada para las variables y parámetros de PWO;
2. una explicación paso a paso de las ecuaciones que generan valores reales;

3. un ruteo numérico reproducible de una actualización de explotación;
4. un mapeo entre ecuaciones y código oficial;
5. una lista de diferencias que requieren decisiones de implementación antes de la experimentación final.

2. Antecedentes y alcance

2.1 Metaheurísticas poblacionales

Sea un problema de minimización con d variables reales:

$$\min_{x \in \mathbb{R}^d} f(x)$$

sujeto a:

$$lb_j \leq x_j \leq ub_j, j=1, \dots, d.$$

Un algoritmo poblacional mantiene N soluciones:

$$X(t) = [X_1(t), X_2(t), \dots, X_N(t)]^T.$$

Cada agente es un vector:

$$X_i(t) = [X_{i,1}(t), X_{i,2}(t), \dots, X_{i,d}(t)].$$

El elemento $X_{i,j}(t)$ es el valor real de la variable j de la solución i en la iteración t . La población se evalúa mediante $f(X_i(t))$. El paper denomina alfa a la mejor solución encontrada. Las implementaciones conservan ese registro entre iteraciones, por lo que en este informe el alfa se trata como el mejor histórico y no necesariamente como el mejor agente de la población actual.

2.2 Contexto del algoritmo

El paper se inspira en la búsqueda cooperativa y en el rally previo a la caza de los licaones africanos. Durante ese rally, los integrantes de la manada expresan su disposición a participar y la influencia del alfa modifica la decisión colectiva [1]. El autor traduce este comportamiento a una selección entre dos modos: continuar buscando, representado como exploración, o iniciar una actualización con referencia en el alfa, representada como explotación.

La correspondencia computacional es directa: la manada es una población, cada licaón es una solución, su posición es un vector de variables, el alfa es la mejor solución registrada y el rally selecciona la ecuación de movimiento. Esta traducción permite conservar el contexto del método sin confundir la metáfora con el mecanismo matemático.

2.3 Trabajo seleccionado

El trabajo principal es *Painted Wolf Optimization: A Novel Nature-Inspired Metaheuristic Algorithm for Real-World Optimization Problems*, publicado el 12 de marzo de 2026 [1]. La selección se justificó por su actualidad, la disponibilidad de código oficial en Python y MATLAB [2] y la presencia explícita de ecuaciones de movimiento en el dominio continuo.

En este informe, **PWO** se refiere exclusivamente a la metaheurística continua y estocástica estudiada en [1].

2.4 Alcance del avance

Este informe se concentra en el dominio de los reales. No se propone todavía una función de transferencia binaria, un operador discreto ni una estrategia de reparación para problemas combinatorios. La fase experimental también

es preliminar: se verifica que una implementación se ejecuta y reduce el fitness en una función sencilla, pero no se pretende validar aún la superioridad estadística reportada por el autor.

3. Método Painted Wolf Optimization

3.1 Inicialización de la población

El pseudocódigo del paper indica que la población se inicializa con posiciones aleatorias. Las implementaciones concretan esa instrucción, para cada agente i y cada variable j , mediante:

$$X_{i,j}(0) = lb_j + u_{i,j}(ub_j - lb_j), u_{i,j} \sim U(0, 1). \quad (1)$$

Esta es la primera asignación de valores a las variables. Si los límites son escalares, se repiten en todas las dimensiones; si son vectores, cada variable utiliza su propio intervalo.

Después de inicializar, el algoritmo evalúa:

$$F_i(t) = f(X_i(t)). \quad (2)$$

La mejor solución histórica hasta la iteración t se define como:

$$(\tau^*, i^*) \in \arg \min_{\substack{0 \leq \tau \leq t \\ 1 \leq i \leq N}} F_i(\tau), X_{\alpha}(t) = X_{i^*}(\tau^*). \quad (3)$$

El código conserva el valor correspondiente en Alpha_score, su vector en Alpha_pos y el índice del agente que obtuvo el registro en Alpha_index. Después de un movimiento, Alpha_pos puede dejar de coincidir con la posición actual de ese agente.

3.2 Parámetro de control e influencia del alfa

El artículo señala que el parámetro $a(t)$ decrece linealmente desde 2 hasta 0. El código Python implementa esa variación como:

$$a(t) = 2 \left(1 - \frac{t}{T} \right), \quad (4)$$

donde T es el máximo de iteraciones.

Con una constante de voto $c = VOTE_{INCREMENT} = 0.04$, la influencia del alfa es:

$$L(t) = \left| \frac{c}{a(t)} \right|. \quad (5)$$

Este valor participa tanto en la votación como en las ecuaciones de movimiento. A medida que $a(t)$ disminuye, $L(t)$ aumenta. Por ello no debe interpretarse $a(t)$ aisladamente como un simple tamaño de paso: el comportamiento resulta de su interacción con el resto de los términos.

3.3 Rally y selección del modo de búsqueda

La formulación indica que cada agente no alfa emite un voto si:

$$F_i(t) - L(t) F_{\alpha}(t) \leq F_{\alpha}(t). \quad (6)$$

La fuerza del rally es:

$$R(t) = c \sum_{i \neq \alpha} I[F_i(t) - L(t)F_\alpha(t) \leq F_\alpha(t)]. \quad (7)$$

El umbral publicado, con r_3 y r_4 uniformes en $[0,1]$, es:

$$H(t) = \text{round} \left(\frac{a(t)(2r_3 - r_4)}{L(t)} + r_4 \right). \quad (8)$$

La decisión es global para la iteración:

$$\begin{cases} R(t) < H(t), & \text{exploración,} \\ R(t) \geq H(t), & \text{explotación.} \end{cases} \quad (9)$$

Cuando corresponde explorar, un número aleatorio $q \sim U(0,1)$ selecciona una de dos estrategias con probabilidad 0.5. En el pseudocódigo, esta selección ocurre dentro de la actualización de cada agente; en las implementaciones, q se vuelve a generar dentro del ciclo de dimensiones.

3.4 Exploración 1: seguimiento de un agente aleatorio

Se selecciona aleatoriamente una solución $X_{rand}(t)$. Para cada dimensión j , se calcula:

$$D_{i,j}^{rand}(t) = \left| (2L(t)r_1 + r_2)X_{rand,j}(t) - X_{i,j}(t) \right|, \quad (10)$$

con $r_1, r_2 \sim U(0,1)$.

Luego:

$$X_i(t+1) = X_{rand,j}(t) - (2a(t)r_1 - a(t))D_{i,j}^{rand}(t). \quad (11)$$

La ecuación (11) produce un escalar a partir de la dimensión j y lo asigna al vector completo X_i . El paper describe esta actualización escalar-a-vector como un mecanismo para que la información de una dimensión influya en la dirección global de búsqueda. En una lectura literal, todas las variables del agente reciben el mismo valor escalar en cada ejecución de la ecuación.

3.5 Exploración 2: coordinación con alfa y media poblacional

La segunda estrategia usa el alfa y la media de la población:

$$X_i(t+1) = (X_\alpha(t) - \bar{X}(t)) - R(t) |X_\alpha(t) - X_i(t)|, \quad (12)$$

donde:

$$\bar{X}(t) = \frac{1}{N} \sum_{i=1}^N X_i(t). \quad (13)$$

La resta $X_\alpha - \bar{X}$ puede interpretarse como una dirección global entre el mejor registro y el centro de la población. El segundo término ajusta cada componente según la distancia absoluta del agente al alfa, ponderada por la fuerza del rally. Esta lectura geométrica es una interpretación del equipo a partir de la ecuación (12).

3.6 Explotación: actualización con referencia en el alfa

El paper denomina explotación a la rama activada cuando $R(t) \geq H(t)$ y la describe como una convergencia hacia el alfa. La ecuación toma el alfa como referencia, aunque un movimiento individual no tiene garantizado reducir su distancia:

$$D_{i,j}^{\alpha}(t) = |X_{\alpha,j}(t) - X_{i,j}(t)|, \quad (14)$$

$$A_1(t) = 2a(t)r_1 - a(t), \quad (15)$$

$$A_2(t) = R(t) + a(t)A_1(t)L(t), \quad (16)$$

$$X_{i,j}(t+1) = X_{\alpha,j}(t) - A_2(t)D_{i,j}^{\alpha}(t). \quad (17)$$

La ecuación (17) muestra directamente la asignación requerida:

1. se toma el valor de la variable j de la mejor solución;
2. se calcula la distancia entre ese valor y el del agente;
3. se pondera la distancia mediante $A_2(t)$;
4. se resta la perturbación al valor del alfa.

La actualización se repite para todas las dimensiones y agentes.

3.7 Factibilidad y reparación del dominio

Las ecuaciones pueden producir valores fuera del intervalo permitido. La implementación oficial utiliza saturación:

$$X_{i,j} \leftarrow \min(ub_j, \max(lb_j, X_{i,j})). \quad (18)$$

Si una variable excede el límite superior, se reemplaza por ub_j ; si cae por debajo del límite inferior, se reemplaza por lb_j . Para las funciones continuas sin otras restricciones, esta operación recupera la factibilidad.

3.8 Pseudocódigo operacional de referencia

El Algoritmo 1 resume el orden común observado en las implementaciones revisadas. Mantiene el ciclo interno sobre las variables: las ramas de exploración escriben el vector completo desde ese ciclo, mientras la explotación actualiza una componente. Las diferencias con el pseudocódigo publicado y entre los índices de Python y MATLAB se detallan en la sección 5.2.

Algoritmo 1. Pseudocódigo operacional de PWO

```

1:  Inicializar N soluciones reales dentro de [lb, ub]
2:  Inicializar Alpha_score como infinito y Alpha_pos
3:  Para cada iteración t hacer
4:    Para cada agente i hacer
5:      Reparar  $X_i(t)$  dentro de [lb, ub]
6:      Evaluar  $F_i(t)$  como  $f(X_i(t))$ 
7:      Si  $F_i(t) < \text{Alpha\_score}$  entonces
8:        Actualizar Alpha_score, Alpha_pos y Alpha_index
9:      Fin si
10:   Fin para
11:   Calcular  $a(t)$  y  $L(t)$ 
12:   Calcular los votos y la fuerza  $R(t)$ 
13:   Generar el umbral  $H(t)$ 
14:   Para cada agente i hacer
15:     Para cada variable j hacer
16:       Generar  $r_1, r_2$  y calcular  $A_1(t)$ 
17:       Si  $R(t) < H(t)$  entonces
18:         Generar  $q \sim U(0, 1)$ 
19:         Si  $q < 0.5$  entonces
20:           Actualizar  $X_i$  completa mediante exploración 1, ecuaciones (10) y (11)
21:         En otro caso

```

```

22:      Actualizar X_i completa mediante exploración 2, ecuación (12)
23:      Fin si
24:      En otro caso
25:      Actualizar X_ij mediante explotación, ecuaciones (14) a (17)
26:      Fin si
27:      Fin para
28:      Fin para
29:      Registrar Alpha_score en la curva de convergencia
30:      Fin para
31:      Retornar Alpha_pos, Alpha_score y la curva de convergencia

```

4. Ruteo numérico de una iteración

Se considera la función Sphere:

$$f(x) = x_1^2 + x_2^2, x \in [-10, 10]^2. \quad (19)$$

Tabla 1

Población inicial utilizada en el ruteo numérico.

Agente	$X_i(t)$	Fitness
1	(1.01, 1.00)	2.0201
2	(0.99, 1.01)	2.0002
3	(1.00, 0.99)	1.9801

El agente 3 es el alfa:

$$X_\alpha(t) = (1.00, 0.99).$$

Se fija $T = 10$, $t = 1$ y $c = 0.04$:

$$a(t) = 2 \left(1 - \frac{1}{10} \right) = 1.8,$$

$$L(t) = \frac{0.04}{1.8} = 0.022222.$$

La condición de voto equivale a:

$$F_i(t) \leq (1 + L(t)) F_\alpha(t) = 2.024102.$$

Los agentes 1 y 2 votan, por lo que:

$$R(t) = 2(0.04) = 0.08.$$

Para hacer el ejemplo reproducible se fijan $r_3 = 0.2$ y $r_4 = 0.4$:

$$2r_3 - r_4 = 0, H(t) = \text{round}(0.4) = 0.$$

Como $R(t) = 0.08 \geq H(t) = 0$, se selecciona explotación.

Para actualizar el agente 1 se fija $r_1 = 0.7$:

$$A_1 = 2(1.8)(0.7) - 1.8 = 0.72,$$

$$A_2 = 0.08 + (1.8)(0.72)(0.022222) = 0.1088.$$

Primera variable:

$$D_{1,1}^\alpha = |1.00 - 1.01| = 0.01,$$

$$X_{1,1}(t+1) = 1.00 - (0.1088)(0.01) = 0.998912.$$

Segunda variable:

$$D_{1,2}^\alpha = |0.99 - 1.00| = 0.01,$$

$$X_{1,2}(t+1) = 0.99 - (0.1088)(0.01) = 0.988912.$$

La nueva solución es:

$$X_1(t+1) = (0.998912, 0.988912).$$

Su fitness es:

$$f(X_1(t+1)) = 1.975772.$$

El valor mejora tanto el fitness previo del agente 1, 2.0201, como el fitness del alfa previo, 1.9801. En la siguiente evaluación, el agente 1 pasaría a ser la mejor solución. Ninguna variable requiere reparación porque ambas permanecen en $[-10, 10]$.

5. Resultados preliminares y reproducibilidad

5.1 Prueba de funcionamiento

Se ejecutó la implementación Python oficial con:

- función Sphere;
- dimensión $d = 3$;
- dominio $[-100, 100]^3$;
- 20 agentes;
- 100 iteraciones;
- semilla NumPy 20260728.

El mejor fitness registrado disminuyó desde 4281.3964 en la primera evaluación hasta 2.5198×10^{-92} en la iteración 100. La curva fue monótona no creciente porque Alpha_score conserva el mejor valor histórico. Una nueva ejecución interna con la misma configuración reprodujo ambos valores y confirmó que el fitness de best_position coincide con best_score.

Este resultado solo demuestra que el código Python se ejecuta y reduce el fitness en una función sencilla con una semilla concreta. No permite afirmar que PWO supere a otros métodos ni reproduce el conjunto de experimentos del artículo.

5.2 Diferencias entre paper y código

La inspección del paper y del repositorio oficial identificó los siguientes puntos:

1. **Umbral del rally.** La ecuación (8) usa dos números aleatorios y reutiliza r_4 . Python y MATLAB generan dos números para $E0$ y un tercer número independiente para el término sumado antes de redondear.
2. **Último valor de $a(t)$.** Python recorre $t=0, \dots, T-1$, por lo que $a(t)$ no llega exactamente a cero. MATLAB recorre $t=1, \dots, T$, haciendo $a(T)=0$ y exponiendo la última iteración a una división por cero.
3. **Parámetro de influencia.** La ecuación de explotación escribe $Linf(t)$ sin definirlo. La tabla de parámetros del paper registra $Linf = 0.05$, mientras la sección del rally fija $VOTE_INCREMENT = 0.04$. El código calcula $Alpha_influence = \text{abs}(0.04/a)$ y usa ese valor en la explotación; el informe lo representa como $L(t)$.
4. **Momento de evaluación.** El pseudocódigo evalúa después del movimiento. Las implementaciones evalúan al comienzo de la iteración siguiente; por ello, la última población generada no vuelve a evaluarse.
5. **Actualización vectorial y selección de estrategia.** Las dos ramas de exploración escriben el vector completo dentro del ciclo sobre j . La segunda estrategia no utiliza j , pero puede ejecutarse repetidamente. Además, el código vuelve a generar q en cada dimensión, por lo que un mismo agente puede alternar entre ambas estrategias dentro de una iteración.
6. **Alfa histórico e índice excluido.** $Alpha_pos$ conserva el mejor vector histórico y $Alpha_index$ identifica al agente que lo encontró. Ese agente se excluye de la votación incluso si una exploración posterior separó su posición actual de $Alpha_pos$.
7. **Reparación.** Los límites se aplican antes de evaluar, no inmediatamente después de cada movimiento.

Estas diferencias no se corrigen silenciosamente en el avance. El paper y el código se tratan como fuentes distintas: las ecuaciones publicadas describen la formulación y la versión Python permite observar una ejecución concreta. La versión definitiva del proyecto deberá declarar cada decisión de implementación.

5.3 Límites de la verificación

El corpus permite revisar por completo el paper principal, inspeccionar las implementaciones en Python y MATLAB y reproducir la prueba Python sobre Sphere. La implementación MATLAB no se ejecutó porque el entorno local no dispone de MATLAB ni Octave. Por esta razón, la equivalencia numérica entre ambas implementaciones queda fuera del alcance de esta verificación.

6. Discusión

PWO combina una decisión poblacional con tres perturbaciones diferentes. La fuerza y el umbral del rally no asignan directamente valores a las variables; seleccionan la regla que realizará esa asignación. La primera regla de exploración se apoya en una solución aleatoria y puede separar la búsqueda de la mejor solución actual. La segunda combina información global, mediante alfa y la media, con la distancia individual. La explotación actualiza cada variable respecto del alfa y es la regla más fácil de seguir en un ruteo manual.

La actualización escalar-a-vector de la primera exploración difiere de una actualización componente a componente. También plantea una pregunta de reproducibilidad porque el código ejecuta esa escritura dentro del ciclo de dimensiones. La experimentación futura deberá comparar al menos la interpretación literal del código con una versión que ejecute cada actualización vectorial una vez por agente.

Del análisis algebraico del rally se infiere que, debido a la división por $L(t)$, el umbral puede tomar valores positivos o negativos de magnitud elevada. Además, $Alpha_score$ participa directamente en la condición de voto, por lo que una transformación de signo o escala de la función objetivo puede modificar la decisión entre exploración y explotación. Estas son inferencias del equipo y requieren evaluación experimental.

Estas observaciones delimitan el trabajo requerido para una reproducción rigurosa. Antes de binarizar PWO conviene estabilizar una especificación continua, probarla sobre funciones de referencia y registrar semillas, parámetros y número de evaluaciones.

7. Trabajo futuro

Para el informe final se proponen cuatro etapas:

1. **Estabilización continua.** Elegir y documentar una interpretación del umbral, del calendario de $a(t)$ y de las actualizaciones vectoriales.
2. **Reproducción.** Comparar resultados básicos con el código oficial sobre funciones Sphere, Rastrigin, Ackley y un subconjunto de las funciones usadas en el artículo.
3. **Binarización.** Transformar las posiciones reales en probabilidades o decisiones binarias mediante una función de transferencia y una regla de muestreo.
4. **Problema discreto.** Definir función objetivo, factibilidad y reparación para un problema como Set Covering Problem o mochila, y comparar contra baselines.

La binarización deberá preservar la separación entre metaheurística y problema: PWO generará perturbaciones; el mecanismo binario representará decisiones; y el solver evaluará y reparará soluciones según las restricciones específicas.

8. Conclusiones

El avance permitió traducir PWO desde su metáfora biológica a un proceso de asignación de valores. Cada lobo corresponde a una solución real, el alfa es la mejor solución histórica y el rally selecciona entre dos movimientos de exploración y uno de explotación. Las ecuaciones (11), (12) y (17) son el núcleo que genera $X_i(t+1)$.

El ruteo muestra que la actualización de explotación puede seguirse variable por variable y que, en el ejemplo construido, mejora el fitness. La ejecución preliminar reproduce una reducción del mejor fitness en Sphere. Sin embargo, el contraste entre paper, pseudocódigo, Python y MATLAB revela diferencias que requieren decisiones antes de presentar resultados experimentales comparativos.

Por tanto, el resultado principal del avance no es todavía una versión binaria ni una afirmación de superioridad, sino una comprensión operacional y trazable del mecanismo continuo. Esta base permite abordar la implementación, reproducción y binarización con decisiones explícitas.

Declaración de uso de inteligencia artificial

Se utilizaron herramientas de inteligencia artificial generativa como apoyo para organizar antecedentes, estructurar borradores y revisar la claridad del texto. Las ecuaciones del método se contrastaron con el paper principal y el código disponible; los resultados de Sphere se reprodujeron con la implementación Python. La versión MATLAB se inspeccionó, pero no se ejecutó en esta revisión. Los autores son responsables de la revisión final y del contenido presentado.

Referencias

1. Sheikhi S. Painted wolf optimization: a novel nature-inspired metaheuristic algorithm for real-world optimization problems. *Comput Mater Contin.* 2026;87(2). doi:[10.32604/cmc.2026.077788](https://doi.org/10.32604/cmc.2026.077788).
2. Sheikhi S. Painted-Wolf-Optimization [Internet]. GitHub; 2026. Disponible en: <https://github.com/saeidsheikhi/Painted-Wolf-Optimization>.